

Linear Key Recovery in the BAG-Loong Reference Implementation

Zihan Liu

Institute of Software, Chinese Academy of Sciences

September 25, 2026

Abstract

The BAG-Loong reference implementation samples its secret matrices X and Y from fixed, publicly known subspaces, although the specification calls for secret random supports. This makes the public-key relation $S = HX + Y$, with public H , amenable to Gaussian elimination. Expanding the relation over $\mathbb{F}_2$ and projecting away the support of Y leaves a linear system for X . We prove an exact recovery criterion based on its column rank. All 40 supplied known-answer-test records, covering four parameter sets, satisfy this criterion: two independent solvers recover each X exactly, and the unmodified decapsulation code reproduces the corresponding session keys. For the largest parameter set, the reduced system has 9360 equations in 728 unknowns per column; one elimination solves all 15 columns.

Keywords: rank-metric cryptography; BAG-Loong; public supports; linear algebra; key recovery; implementation analysis.

1 Introduction

BAG-Loong is a proposed rank-metric key-encapsulation mechanism (KEM) based on blended-block errors and augmented Gabidulin codes [1]. Its four parameter sets are labelled 128, 256, 384, and 512. The public key satisfies

$$S = HX + Y, \tag{1}$$

where H is reconstructed from a public seed and X is the secret matrix used for decryption in the underlying public-key encryption (PKE) scheme. The support of a matrix is the binary vector space spanned by its entries. The specification samples X and Y from secret supports with prescribed dimensions and intersection dimension. In the reference implementation, both supports are fixed spans of low-degree monomials. The entries of X and Y depend on a separate key-generation seed, but their support spaces remain fixed.

This choice makes (1) easy to solve. Each entry of X and Y can be expanded in a known basis, giving a linear system over $\mathbb{F}_2$. Projecting onto coordinates outside the support of Y eliminates Y . The remaining system determines X whenever its coefficient matrix has full column rank. All columns share this matrix and can be solved together.

We give an exact criterion for recovery and show that eliminating Y preserves all information about X . The supplied known-answer tests confirm that the attack recovers the decryption matrices and suffices to decapsulate the corresponding ciphertexts. The distinction between the implementation and the specification is central here: the attack uses the particular support spaces chosen by the reference code.

Related work. Solving for error coordinates once their support is known is standard in rank-metric cryptanalysis. Gaborit, Ruatta, and Schrek use this step after choosing a candidate space containing the error support [2, Sections 3.2–3.3]; it also appears after support recovery in LRPC decoding [3, Sections 4.3 and 4.5]. In the implementation examined here, the key generator fixes both relevant supports to public subspaces. The attack therefore starts at the linear solving step, with no support search.

Saarinen reports secret-dependent pivot selection in the Gabidulin decoder of the same source archive [4]. That report concerns local control-flow and cache observations and does not demonstrate key recovery. Our method uses only the public key. Making the decoder constant-time would therefore leave the public-support problem unchanged.

2 Setting and the affected implementation

2.1 Notation and public information

Let $F = \mathbb{F}_{2^m}$ with a fixed public basis over $\mathbb{F}_2$. For $z \in F$, write $[z] \in \mathbb{F}_2^m$ for its coordinate column and $[z]_b$ for coordinate b , where $0 \leq b < m$. The support of a matrix A over F is

$$\text{Supp}(A) = \text{span}_{\mathbb{F}_2} \{A_{ij} : i, j\},$$

and its rank weight is $\dim_{\mathbb{F}_2} \text{Supp}(A)$. We reserve matrix rank for the rank of a linear map over its stated coefficient field.

In (1),

$$H \in F^{n \times n}, \quad X, Y, S \in F^{n \times \ell},$$

where $\ell = n_1$ in the submission’s notation. We reconstruct H from the public seed using the submitted expansion routine and decode S from the public key using the submitted field encoding. The field basis used by the implementation is the polynomial basis $1, \beta, \dots, \beta^{m-1}$ modulo the polynomial listed in Table 1. Let V_X, V_Y denote the support spaces used to sample X, Y , with dimensions d_X, d_Y and intersection dimension g .

Table 1: Parameters and field polynomials used by the examined implementation. d_X, d_Y are support dimensions and $g = \dim(V_X \cap V_Y)$.

Label	m	n	ℓ	d_X	d_Y	g	Polynomial for β
128	47	42	10	5	5	4	$T^{47} + T^5 + 1$
256	67	65	12	5	6	3	$T^{67} + T^5 + T^2 + T + 1$
384	83	83	14	6	6	3	$T^{83} + T^7 + T^4 + T^2 + 1$
512	97	104	15	7	7	4	$T^{97} + T^6 + 1$

2.2 A public pool of support elements

In each reference directory, `src/loong_pke.c` defines `sample_support_pair_logical()`. The four copies of this file are byte-identical. The sampler ignores the reader for the extendable-output function (XOF) when constructing the support pool and sets its i th element to β^i . The pool-construction routine contains the following statements:

```
(void)reader;
(void)require_one_in_v2;
/* Inside the loop, after zeroing candidate: */
rbc_elt_set_coefficient(&candidate, pool_size, 1U);
```

The pool is linearly independent and depends only on public parameters.

Overlapping subsets of this pool form bases for the two support spaces:

$$\begin{aligned} I_X &= \{0, \dots, d_X - 1\}, \\ I_Y &= \{0, \dots, g - 1\} \cup \{d_X, \dots, d_X + d_Y - g - 1\}, \\ V_X &= \text{span}_{\mathbb{F}_2}\{\beta^a : a \in I_X\}, \quad V_Y = \text{span}_{\mathbb{F}_2}\{\beta^b : b \in I_Y\}. \end{aligned} \tag{2}$$

Every entry of X lies in V_X and every entry of Y lies in V_Y . The key-generation seed determines their coordinates in the chosen bases. The key-generation code then computes (1) directly.

For the 128 parameter set, for example,

$$V_X = \text{span}_{\mathbb{F}_2}\{1, \beta, \beta^2, \beta^3, \beta^4\}, \quad V_Y = \text{span}_{\mathbb{F}_2}\{1, \beta, \beta^2, \beta^3, \beta^5\}.$$

The same construction gives the exponents in Table 2. The sampler also serves encryption, but the public-key relation alone suffices for our recovery procedure.

Table 2: Public support exponents obtained from the source code.

Label	I_X	I_Y
128	$\{0, 1, 2, 3, 4\}$	$\{0, 1, 2, 3, 5\}$
256	$\{0, 1, 2, 3, 4\}$	$\{0, 1, 2, 5, 6, 7\}$
384	$\{0, 1, 2, 3, 4, 5\}$	$\{0, 1, 2, 6, 7, 8\}$
512	$\{0, 1, 2, 3, 4, 5, 6\}$	$\{0, 1, 2, 3, 7, 8, 9\}$

3 Linear recovery from known supports

The argument applies to any two known $\mathbb{F}_2$ -subspaces of F ; monomial bases merely simplify the coordinate projection. Fix public bases $u_1, \dots, u_{d_X}$ of V_X and $v_1, \dots, v_{d_Y}$ of V_Y . Let $B_X \in \mathbb{F}_2^{m \times d_X}$ and $B_Y \in \mathbb{F}_2^{m \times d_Y}$ have these basis vectors as coordinate columns. For $h \in F$, define $T(h) \in \mathbb{F}_2^{m \times m}$ by

$$T(h)[z] = [hz] \quad \text{for every } z \in F.$$

This is a binary linear map because multiplication by a fixed field element is linear over $\mathbb{F}_2$.

3.1 The full linear system

Consider one column $s = Hx + y$. Express its entries as

$$x_k = \sum_{a=1}^{d_X} \alpha_{k,a} u_a, \quad y_i = \sum_{b=1}^{d_Y} \gamma_{i,b} v_b, \quad \alpha_{k,a}, \gamma_{i,b} \in \mathbb{F}_2.$$

Collect these unknowns in the columns $\alpha_k \in \mathbb{F}_2^{d_X}$ and $\gamma_i \in \mathbb{F}_2^{d_Y}$. For each $1 \leq i \leq n$, the field equation becomes

$$\sum_{k=1}^n T(H_{ik}) B_X \alpha_k + B_Y \gamma_i = [s_i]. \tag{3}$$

Let α stack $\alpha_1, \dots, \alpha_n$ and let γ stack $\gamma_1, \dots, \gamma_n$. Stacking the equations gives a binary system

$$\mathcal{A}_H \begin{bmatrix} \alpha \\ \gamma \end{bmatrix} = [s], \quad \mathcal{A}_H = [\mathcal{B}_H \quad I_n \otimes B_Y], \tag{4}$$

where block (i, k) of $\mathcal{B}_H$ is $T(H_{ik}) B_X$ and $[s]$ stacks the coordinates of $s_1, \dots, s_n$. It has nm equations and $n(d_X + d_Y)$ unknowns. All entries of $\mathcal{A}_H$ and its right-hand side are obtained from public information.

3.2 Eliminating Y

Choose a full-row-rank matrix $P \in \mathbb{F}_2^{(m-d_Y) \times m}$ such that

$$\ker P = \text{im } B_Y.$$

Such a matrix is obtained by elementary linear algebra from the known basis of V_Y . Applying P to (3) eliminates γ_i :

$$\sum_{k=1}^n PT(H_{ik})B_X\alpha_k = P[s_i]. \quad (5)$$

Define M_H to be the block matrix with blocks $PT(H_{ik})B_X$, and let r_s stack the $P[s_i]$. The reduced system is

$$M_H\alpha = r_s, \quad M_H \in \mathbb{F}_2^{n(m-d_Y) \times nd_X}. \quad (6)$$

It has $n(m - d_Y)$ equations and nd_X unknowns.

In BAG-Loong, P simply retains the coordinates outside I_Y . For the 128 parameter set, it retains coordinates 4, 6, 7, ..., 46. Equation (5) then reads

$$\sum_{k=1}^{42} \sum_{a=1}^5 [H_{ik}\beta^{a-1}]_b \alpha_{k,a} = [s_i]_b, \quad b \in \{4, 6, 7, \dots, 46\}. \quad (7)$$

Thus a single column requires solving 1764 equations in 210 unknowns. Whether these equations determine the column uniquely depends on the rank of M_H , as the following theorem makes precise.

Theorem 1 (Exact recovery criterion). *Fix H, V_X, V_Y as above. The following conditions are equivalent:*

- (i) M_H has rank nd_X over $\mathbb{F}_2$.
- (ii) For every $s \in HV_X^n + V_Y^n$, there is exactly one pair $(x, y) \in V_X^n \times V_Y^n$ with $s = Hx + y$.
- (iii) $\mathcal{A}_H$ has rank $n(d_X + d_Y)$ over $\mathbb{F}_2$.

Moreover,

$$\dim \ker \mathcal{A}_H = \dim \ker M_H, \quad \text{rank } \mathcal{A}_H = nd_Y + \text{rank } M_H. \quad (8)$$

Under these conditions, binary Gaussian elimination on (6) recovers x and then $y = s - Hx$.

Proof. An element α lies in $\ker M_H$ if and only if the corresponding $x \in V_X^n$ satisfies $Hx \in V_Y^n$. For each such x , the unique $y \in V_Y^n$ with $Hx + y = 0$ is $y = -Hx$. Since the entries of y have unique coordinates in the basis $v_1, \dots, v_{d_Y}$, this defines a linear bijection between $\ker M_H$ and $\ker \mathcal{A}_H$. Their dimensions are therefore equal, and rank-nullity gives the rank identity.

For any consistent right-hand side, the solution set is a translate of the kernel, so the solution is unique if and only if the kernel is trivial. This proves the equivalence. Finally, a solution to the reduced system satisfies $P([s_i] - [(Hx)_i]) = 0$ for every i , so $s - Hx \in V_Y^n$. It therefore gives a solution of the original equation with $y \in V_Y^n$. $\square$

If M_H is rank deficient, further rank-weight constraints may narrow the solution set within the stated support spaces. We did not need such constraints for the tested keys.

3.3 Explicit recovery algorithm

Theorem 1 gives the uniqueness criterion. The following procedure computes the solution from the public key and the public support bases identified in Section 2. It handles all ℓ columns at once. Set $r = n(m - d_Y)$ and $c = nd_X$; all matrix operations below, except the field-element products and reconstruction, are over $\mathbb{F}_2$.

1. **Read the public data and choose the projection.** Reconstruct H and decode S from the public key. In this implementation, form P from rows $b+1$ of I_m , for $b \in \{0, \dots, m-1\} \setminus I_Y$ in increasing order. For general known supports, compute a basis of $\ker(B_Y^T)$ and use the transposed basis vectors as the rows of P . Since B_Y has full column rank, this gives $\ker P = \text{im } B_Y$.
2. **Build the binary equations.** Construct $M_H \in \mathbb{F}_2^{r \times c}$ and $R \in \mathbb{F}_2^{r \times \ell}$ entry by entry as

$$(M_H)_{(i,q),(k,a)} = (P[H_{ik}u_a])_q, \quad R_{(i,q),j} = (P[S_{ij}])_q.$$

Here $1 \leq i, k \leq n$, $1 \leq q \leq m - d_Y$, $1 \leq a \leq d_X$, and $1 \leq j \leq \ell$; row pairs (i, q) and column pairs (k, a) are ordered lexicographically. In particular, column (k, a) has number $(k-1)d_X + a$. These entries require only public field elements and the known support bases.

3. **Eliminate and read the coordinates.** Apply Gauss–Jordan elimination to $[M_H \mid R]$, choosing pivots only from the first c columns. Start with $t = 0$ and scan these columns from left to right. In the current column, find a row below the first t pivot rows with entry 1. If there is none, move to the next column. Otherwise, swap that row into position $t+1$ and XOR it into every other row whose entry in this column is 1; then increment t . Every swap and XOR acts on the entire augmented row, including all ℓ right-hand sides.

After all c columns have been scanned, if a zero coefficient row has any nonzero right-hand-side entry, report inconsistency and stop. If no such row exists but $t < c$, report that these equations do not determine X uniquely and stop. Otherwise, the resulting matrix has the form

$$[M_H \mid R] \longrightarrow \begin{bmatrix} I_c & A \\ 0 & 0 \end{bmatrix}, \quad A \in \mathbb{F}_2^{c \times \ell}.$$

Thus the first c rows of the transformed right-hand side give A directly. Each pivot is already 1, so no division is needed.

4. **Reconstruct the field matrices.** For $1 \leq k \leq n$ and $1 \leq j \leq \ell$, set

$$X_{kj} = \sum_{a=1}^{d_X} A_{(k-1)d_X+a,j} u_a, \quad Y = S - HX,$$

and output (X, Y) .

For a key generated with the stated supports, the equations are consistent. If the elimination finds c pivots, it solves $M_H A = R$, and Theorem 1 ensures that the reconstructed matrices are the unique pair in the prescribed support spaces. The same row operations solve every column, so the coefficient matrix is eliminated only once.

3.4 Computational cost

With the dimensions r, c defined above, straightforward elimination on the augmented matrix $[M_H \mid R]$ costs $O(rc^2 + rcl)$ binary operations when $r \geq c$. Packing rows into machine words speeds up the row additions. Constructing M_H requires the coordinates of the $n^2 d_X$ products $H_{ik}u_a$. Since each u_a in this implementation is a low-degree power of β , these products can be computed by shifts and reduction modulo the field polynomial.

Table 3 lists both system sizes. In the experiments below, we compute the rank of M_H for each seeded public matrix H ; Appendix A gives a probabilistic bound for comparison under an independent, uniform model for the entries of H .

Table 3: Linear systems for one column, over $\mathbb{F}_2$. All columns share the coefficient matrix.

Label	Full system		Reduced system		Columns
	Equations	Unknowns	Equations	Unknowns	
128	1974	420	1764	210	10
256	4355	715	3965	325	12
384	6889	996	6391	498	14
512	10088	1456	9360	728	15

4 Experimental evaluation

4.1 Test setting

We use the BAG-Loong archive retrieved on September 25, 2026 from the submission website [1]. Its SHA-256 digest is

2795fd57d3d00791652256362b4a21982116d3aad1fb63633dac923e0e76e05b.

For each of the four parameter sets, we generated ten deterministic known-answer-test (KAT) records using the reference implementation. We verified that they matched the supplied test vectors byte for byte and that encapsulation and decapsulation produced the same session keys. Our experiments cover these 40 records; they do not estimate the recovery probability for randomly generated keys.

4.2 Independent recovery implementations

We implemented both linear systems independently, using SageMath for the full system and a separate Python solver for the reduced one. In each case, the reference secret matrix serves only to check the recovered result.

Full-system solver. We reconstruct H and decode S from each public key using the reference implementation’s public expansion and coordinate representation. The SageMath solver constructs (4) using only H, S and the public support bases, computes the rank, and solves for all columns. The reference X is used only to compare against the recovered matrix; it enters neither the coefficient matrix nor the right-hand sides.

Reduced-system solver. An independent Python implementation uses (6), represents binary rows as integers, and eliminates by XOR. Its input consists only of the public parameters, H , and S . It solves all ℓ columns together and checks the projected equations. Only after X has been recovered does a separate comparison step read the reference secret keys and session keys.

Both implementations recovered the reference X exactly for all 40 records, with zero-dimensional kernels in every case. The reduced matrices had ranks 210, 325, 498, 728, respectively, for every record in the corresponding parameter set. By Theorem 1, the full systems consequently have ranks 420, 715, 996, 1456, agreeing with the direct Sage computations.

4.3 Session-key recovery and the KEM secret key

Recovering X leaves one part of the serialized KEM secret key unknown: the independent rejection value ξ . The valid-ciphertext branch of decapsulation uses a message-derived value in its place. We therefore tested whether the recovered matrix suffices by constructing a KEM

secret key from the public seed, recovered X , the public key, and an arbitrary ξ , then calling the unmodified official decapsulation function on each KAT ciphertext.

For all 40 records, decapsulation succeeded and returned the reference session key with ξ filled with either 0x00 or 0xA5 bytes. Replacing X with zero, as a negative control, caused rejection and a session-key mismatch in every test.

4.4 Measured cost

The experiments ran on an arm64 macOS host with Python 3.12.3, SageMath 10.5, and Apple clang 21.0.0, without dedicated performance isolation. Table 4 gives the observed recovery times for the ten records in each parameter set, with all columns of X recovered in each run.

Table 4: Results on the supplied KAT corpus. Times are in seconds to recover one secret matrix. The Sage column gives the observed range; the Python column gives the mean.

Label	X matches	Session keys	Kernel dimension	Sage	Python
128	10/10	10/10	0	0.6–1.1	0.049
256	10/10	10/10	0	2.0–2.1	0.171
384	10/10	10/10	0	4.6–4.8	0.432
512	10/10	10/10	0	9.6–10.3	0.988

The timings measure the two recovery procedures, whose representations and workloads differ; they do not isolate the effect of programming language. The Sage timing covers binary-matrix construction, rank computation, solving all right-hand sides, and comparison with the reference X . It excludes process startup, field parsing, public expansion, and the earlier diagnostic computation of Y from the reference X . The Python timing covers reduced-matrix construction, elimination, checking the projected equations, and serializing X . It excludes compilation, public expansion, file I/O, and C decapsulation.

5 Scope and implications

5.1 Consequences of public supports

Changing the key-generation seed changes the entries of X and Y , but leaves their public support spaces unchanged. The new public key therefore yields linear equations for their binary coordinates in just the same way, with recovery certified by the rank test in Theorem 1. Since any holder of the public key can expand H , the attack requires no weakness in the hash function.

5.2 Relation to the specification and to repairs

The specification calls for secret random supports rather than the fixed monomial spaces found in Section 2. Keeping these spaces secret would prevent construction of M_H by our method, although the resulting implementation would still require security analysis and validation of its sampler and decoder.

The sampling conditions also affect correctness. KeyGen requires $1 \in \text{Supp}(Y)$, and encryption requires $\text{Supp}(E) \subseteq \text{Supp}(R_2)$. Replacing the fixed pool with random elements does not enforce these conditions. Appendix B examines this modification and its decoding failures, which concern a sampler that still differs from the specification.

6 Conclusion

The reference sampler makes the support spaces needed for linear key recovery public. Once they are known, the attack reduces to binary elimination, with full column rank providing an

exact certificate that the recovered X is unique. On the supplied test corpus, this yields both the decryption matrices and the valid-ciphertext session keys. The support-sampling discrepancy is therefore enough to compromise the examined implementation. A repair must account for the specification’s sampling conditions and the decoder’s behaviour on the resulting errors.

References

- [1] BAG-Loong submission team. *Loong with Blended-block Errors and Augmented Gabidulin Codes (BAG-Loong)*. Algorithm specification, reference implementation, and test vectors. Next-generation Commercial Cryptographic Algorithms Program, Round 1 submission package, 2026. Retrieved September 25, 2026. <https://www.niccs.org.cn/niccs/Proposal/Public-Key%20Cryptographic%20Algorithms/Round%201%20candidates/BAG-Loong.zip>.
- [2] Philippe Gaborit, Olivier Ruatta, and Julien Schrek. *On the Complexity of the Rank Syndrome Decoding Problem*. Author preprint, arXiv:1301.1026, 2013. <https://arxiv.org/abs/1301.1026>.
- [3] Nicolas Aragon, Philippe Gaborit, Adrien Hauteville, Olivier Ruatta, and Gilles Zémor. *Low Rank Parity Check Codes: New Decoding Algorithms and Applications to Cryptography*. Author preprint, arXiv:1904.00357, 2019. <https://arxiv.org/abs/1904.00357>.
- [4] Markku-Juhani O. Saarinen. *BAG-Loong (kem-03): Secret-dependent pivoting in the Gabidulin decoder*. NGCC implementation analysis, September 23, 2026. <https://ngcc.dev/reports/kem-03.html>.

A An idealized rank bound

Fix V_X, V_Y and sample every entry of H independently and uniformly from F . This idealized model gives a bound on rank deficiency; applying it to the implementation’s public-seed expansion would require additional assumptions.

Proposition 2. *When the entries of H are independent and uniform in F ,*

$$\Pr_H[\text{rank } M_H < nd_X] \leq \min\left\{1, (2^{nd_X} - 1)2^{-n(m-d_Y)}\right\}. \quad (9)$$

In particular, if $d_X + d_Y < m$, this is strictly smaller than $2^{-n(m-d_X-d_Y)}$.

Proof. By the kernel characterization in Theorem 1, rank deficiency occurs precisely when some nonzero $x \in V_X^n$ satisfies $Hx \in V_Y^n$. Fix any nonzero $x \in V_X^n$ and choose an index k with $x_k \neq 0$. For each row i , condition on all H_{ij} with $j \neq k$. The value

$$(Hx)_i = H_{ik}x_k + \sum_{j \neq k} H_{ij}x_j$$

is uniform in F , because multiplication by x_k is a bijection and H_{ik} remains uniform under this conditioning. Different rows use independent random variables. Therefore

$$\Pr_H[Hx \in V_Y^n] = \prod_{i=1}^n \frac{|V_Y|}{|F|} = 2^{-n(m-d_Y)}.$$

The union bound, explicitly summing over every nonzero candidate, gives

$$\begin{aligned} \Pr_H[\text{rank } M_H < nd_X] &\leq \sum_{x \in V_X^n \setminus \{0\}} \Pr_H[Hx \in V_Y^n] \\ &= (2^{nd_X} - 1)2^{-n(m-d_Y)}. \end{aligned}$$

Combining this estimate with the trivial upper bound of 1 proves (9). The strict bound follows from $2^{nd_X} - 1 < 2^{nd_X}$. $\square$

Only H is random in this argument. The union bound requires no independence between the events indexed by different x , and the proof makes no assumption on the distribution of X or Y .

B Limits of a sampler-only modification

A natural first modification is to replace the fixed monomial pool with random field elements. To interpret the resulting decoding failures, we first derive the error-support bound under the specification’s sampling conditions.

Let V_1, V_2 be the supports of encryption matrices R_1, R_2 . Write $d_1 = \dim V_1$, $d_2 = \dim V_2$, and $g_R = \dim(V_1 \cap V_2)$. For spaces $U, V \subseteq F$, define their product space by

$$UV = \text{span}_{\mathbb{F}_2}\{uv : u \in U, v \in V\}.$$

For encryption matrices $R_1, R_2 \in F^{n_2 \times n}$ and $E \in F^{n_2 \times \ell}$, where n_2 is the ciphertext row count, the PKE decryption error is $R_2Y + E - R_1X$. Under the specification’s conditions $1 \in V_Y$ and $\text{Supp}(E) \subseteq V_2$, the support of the error is contained in $V_2V_Y + V_1V_X$. Indeed, $V_2 \subseteq V_2V_Y$, so adding E does not enlarge that space.

Choose bases of V_X and V_Y that extend a common basis of $V_X \cap V_Y$, and do the same for V_1 and V_2 . Counting the resulting basis products gives

$$\dim(V_2V_Y + V_1V_X) \leq d_2d_Y + d_1d_X - g_Rg. \quad (10)$$

The two lists contain $d_2d_Y + d_1d_X$ basis products, with g_Rg products appearing in both. Removing these repetitions preserves the span; the remaining products need not be linearly independent.

For the four parameter sets, (d_1, d_2, g_R) equals $(5, 5, 3)$, $(6, 6, 3)$, $(7, 7, 4)$, and $(7, 7, 4)$, respectively. Together with Table 1, equation (10) gives the bounds 38, 57, 72, 82, matching the specification. Experiments with a larger error-support model therefore do not establish a violation of these bounds. Whether the submitted decoder handles errors sampled under the stated conditions remains a separate question.

We reproduced the pool-only modification using randomly sampled, linearly independent field elements. The support splitting, coefficient sampling, and decoder were left unchanged. The original code succeeded in all ten deterministic PKE trials for each parameter set. We computed each residual by re-encoding the recovered message and subtracting the resulting codeword from the decoder input. The residual rank weights were 12, 16, 17, 18 for the four parameter sets, respectively.

The modified code failed in all twenty trials per parameter set, returning error -3 in all 80 cases. A traced trial per parameter set confirmed rejection in `recover_support_basis()`: after the collected basis reaches the capacity ε (respectively 33, 51, 66, 73), the next nonzero tail element is rejected before testing whether it is dependent on that basis.

This experiment also exposed a mismatch between the sampled spaces and the generated matrices. The modified sampler still ignores `require_one_in_v2`, while a later sampling step forces the last entry of Y to be 1. In the first trial in each parameter set, the sampled V_Y did not contain 1 and the generated Y had rank weight $d_Y + 1$: respectively 6, 7, 7, 8 instead of 5, 6, 6, 7. The encryption code also samples E from the support of R_1 . These departures from the specification remain after the pool is randomized, so the failures do not settle whether a conforming sampler would work with the submitted decoder.